

# **Competitive Programming**

## **Section 50: Advanced Algorithms Mixed Contest**

Competitive Programming Course

2026-08-14

# Table of contents

|   |                                  |   |
|---|----------------------------------|---|
| 1 | Mixed advanced contest habits    | 3 |
| 2 | Complete twelve-problem set      | 4 |
| 3 | Diagnose the mathematical object | 5 |
| 4 | Proof checkpoints                | 6 |
| 5 | Implementation and testing       | 7 |
| 6 | Exercises                        | 8 |

# **1 Mixed advanced contest habits**

At this level, the danger is applying a powerful tool before the problem model has earned it.

## 2 Complete twelve-problem set

The virtual contest contains twelve problems. A–F establish the broad core; G–L are ordered as a progressively harder second half:

| Category                           | Problem package               |
|------------------------------------|-------------------------------|
| String Algorithms I                | E. Pattern Prefix Matches     |
| String Algorithms II               | F. Suffix LCP Queries         |
| Flows and Matchings                | G. Domino Placement           |
| Rerooting and Advanced Tree DP     | H. Weighted Tree Distances    |
| Meet-in-the-Middle                 | I. Balanced Subset Difference |
| Divide-and-Conquer DP Optimization | J. Monge Partition Table      |
| Convex Hull Trick and Li Chao Tree | K. Online Line DP             |
| Min-cost flow synthesis            | L. Profitable Disjoint Routes |

All twelve packages are local and contain statements, stubs, references, and tests. The checker runs A–L.

### 3 Diagnose the mathematical object

For every local task, write a short model before coding:

```
What is the state, relation, or algebraic constraint?  
Which quantities are invariant under the allowed operations?  
Can the input be split into independent parts or reduced recursively?  
What makes a candidate answer valid?
```

Then derive a small case by hand and compare it with the proposed model. Do not use the local statements as labels for known algorithms; the editorial is where their final reductions belong.

## 4 Proof checkpoints

An advanced solution needs a proof of the transformation as well as a proof of the algorithm. Check that every original solution maps to a constructed object and every constructed object maps back. For a recurrence, prove base cases and the split. For a modular condition, verify existence and uniqueness assumptions before combining constraints.

## 5 Implementation and testing

Use types large enough for intermediate products, normalize modular values, and make base cases explicit. Write a deliberately slow oracle for very small input when possible. Test neutral elements, repeated constraints, impossible cases, and maximum bit lengths separately.

## 6 Exercises

Attempt all twelve problems by recording the model, the key proof obligation, and a tiny counterexample that would disprove your idea. Read each source section's editorial only after that written attempt.